

Policy Optimization: Actor-Critic to PPO

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 6, 2026

1. Recap
2. Learning Outcomes
3. Actor-Critic Methods
4. Better Advantage Estimates (GAE)
5. A2C and A3C
6. Why Big Steps Break Policies
7. The Surrogate Objective
8. Trust Region Policy Optimization (TRPO)
9. Proximal Policy Optimization (PPO)
10. Group Relative Policy Optimization (GRPO)
11. Summary
12. References

Policy Optimization: **Recap (REINFORCE)**

- ▶ **The objective is unchanged from REINFORCE** — maximize the expected discounted return:

$$\mathcal{J}(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid \pi_\theta \right], \quad \theta^* = \arg \max_{\theta} \mathcal{J}(\theta)$$

- ▶ We derived its gradient and, after variance reduction, ended at the estimator:

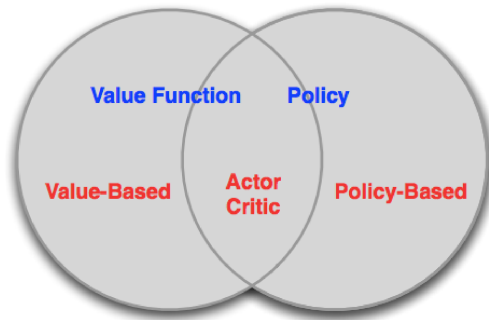
$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} A^{\pi}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- ▶ $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ is the **advantage function** — the best baseline
- ▶ Using A^{π} instead of raw returns reduces variance without introducing bias
- ▶ **The open question we ended on:** how do we actually estimate A^{π} ?

- ▶ **Today we cash that promise.**
- ▶ We learn a critic V_ϕ alongside the policy — this is the **Actor-Critic** architecture
- ▶ Good news: we only need to learn V^π , not Q^π — one network, one output per state
- ▶ The critic is trained by **TD learning** (which we unpack in a few slides); its one-step error is what gives us an online, low-variance estimate of A^π
- ▶ Then we scale up: A2C/A3C $\rightarrow$ trust region (TRPO) $\rightarrow$ clipped objective (PPO)

- ▶ Explain **TD learning** — what the TD error is, why the critic is trained on its *square*, and how it connects to the Q-learning update from value-based methods
- ▶ Explain why a learned critic reduces variance and enables online updates (**Actor-Critic**), and derive the TD-residual estimate of the advantage
- ▶ Use λ (**GAE**) to trade bias against variance, and read $\hat{A}_t$ as “how much better than usual this action turned out”
- ▶ Describe **A2C** and A3C at a high level and identify when each is preferred
- ▶ Explain why we need a **surrogate objective** $\mathcal{L}(\theta) = \mathbb{E}_t[r_t(\theta)\hat{A}_t]$ to score a candidate policy from already-collected data — and why it is only trustworthy nearby
- ▶ State the **TRPO** trust-region constraint, and why its cost is what motivates PPO
- ▶ Implement the **PPO** clipped objective, and identify the three terms of its combined loss

Policy Optimization: **Actor-Critic** Methods



- ▶ **Value-based** methods (Q-learning, DQN): learn V or Q , derive policy implicitly
- ▶ **Policy-based** methods (REINFORCE): learn π_θ directly, no value function
- ▶ **Actor-Critic** sits at the intersection: learn *both* a policy *and* a value function

- From REINFORCE: subtracting a **state-dependent baseline** $b(s)$ from the reward-to-go reduces variance with zero bias:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\underbrace{\sum_{t' \geq t} \gamma^{t'-t} r_{t'}}_{\text{reward-to-go}} - b(s_t) \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- From REINFORCE: subtracting a **state-dependent baseline** $b(s)$ from the reward-to-go reduces variance with zero bias:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \underbrace{\left(\sum_{t' \geq t} \gamma^{t'-t} r_{t'} \right)}_{\text{reward-to-go}} - b(s_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- The reward-to-go is one *sampled* rollout starting from (s_t, a_t) . Its *expectation* is, by definition, the Q-value: $Q^{\pi}(s_t, a_t) = \mathbb{E} \left[\sum_{t' \geq t} \gamma^{t'-t} r_{t'} \mid s_t, a_t, \pi \right]$
- So the same estimator, written in expectation, is:

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_t \left[(Q^{\pi}(s_t, a_t) - b(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

- Best choice of baseline: $b(s) = V^{\pi}(s)$, giving the advantage $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$

- ▶ So the ideal weight on each $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$ is $A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$
- ▶ But we don't know V^{π} — we have to **learn it**
- ▶ This turns the baseline into a **critic**: a second network $V_{\phi}(s)$ trained alongside the actor π_{θ}
- ▶ **Actor** $\pi_{\theta}(a|s)$: selects actions, updated by the policy gradient (as in REINFORCE)
- ▶ **Critic** $V_{\phi}(s)$: evaluates states, updated by **TD learning** — the subject of the next slide

- ▶ So the ideal weight on each $\nabla_{\theta} \log \pi_{\theta}(a_t|s_t)$ is $A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$
- ▶ But we don't know V^{π} — we have to **learn it**
- ▶ This turns the baseline into a **critic**: a second network $V_{\phi}(s)$ trained alongside the actor π_{θ}
- ▶ **Actor** $\pi_{\theta}(a|s)$: selects actions, updated by the policy gradient (as in REINFORCE)
- ▶ **Critic** $V_{\phi}(s)$: evaluates states, updated by **TD learning** — the subject of the next slide
- ▶ **Why only V , and not Q as well?** $A^{\pi} = Q^{\pi} - V^{\pi}$ looks like it needs both. It doesn't: one observed reward plus the critic at the next state already stands in for Q^{π} . That is exactly what the TD error does.

What Is TD Learning? (You Have Already Seen It)

Temporal Difference (TD) learning = update a prediction using a *later, better-informed* prediction, instead of waiting for the final outcome.

- ▶ **Monte Carlo:** to learn “how good is s_t ?”, play the episode to the end and use the actual **reward-to-go** $R_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'}$ — the same quantity REINFORCE used. Correct on average, but you wait, and the answer swings wildly.
- ▶ **TD:** take *one* step. You now know the real reward r_t and you can look up your own estimate of the next state. That combination is already a better estimate of s_t 's value than your current guess:

$$\underbrace{V_{\phi}(s_t)}_{\text{old guess}} \quad \text{vs.} \quad \underbrace{r_t + \gamma V_{\phi}(s_{t+1})}_{\text{TD target: 1 real reward + guess for the rest}}$$

- ▶ The gap between them is the **TD error**:

$$\delta_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$$

- ▶ Using your own later estimate inside the target is called **bootstrapping**.

- ▶ Every value-based update in this course has been a TD update — we simply had not named it:

$$\textbf{Q-learning: } Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

$$\textbf{SARSA: } Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right]$$

$$\textbf{TD for our critic: } V_{\phi}(s_t) \leftarrow V_{\phi}(s_t) + \alpha \left[\underbrace{r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)}_{\delta_t} \right]$$

- ▶ Same shape every time: *move the prediction a little toward the target*. We first called the bracket the **Bellman error**; for a one-step sample it is the TD error.

- ▶ Every value-based update in this course has been a TD update — we simply had not named it:

$$\textbf{Q-learning: } Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

$$\textbf{SARSA: } Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right]$$

$$\textbf{TD for our critic: } V_{\phi}(s_t) \leftarrow V_{\phi}(s_t) + \alpha \left[\underbrace{r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)}_{\delta_t} \right]$$

- ▶ Same shape every time: *move the prediction a little toward the target*. We first called the bracket the **Bellman error**; for a one-step sample it is the TD error.
- ▶ **Why TD suits an actor-critic**: it gives a learning signal at *every* timestep (no waiting for the episode to end), and it works in continuing tasks with no terminal state.
- ▶ **The catch**: the target contains our own guess $V_{\phi}(s_{t+1})$. If the critic is wrong, we are chasing a wrong target — TD is **biased** but **low variance**, the mirror image of Monte Carlo.

Training the Critic: Why the *Squared* Error?

The critic is trained by gradient descent on the *squared* TD error — the DQN loss $(y_i - Q)^2$, applied to V :

$$\mathcal{L}(\phi) = \mathbb{E}_t[\delta_t^2] = \mathbb{E}_t \left[\underbrace{(r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))}_{\text{target } y_t}^2 \right]$$

- **Why not just δ_t ?** It is signed — over-estimates give $\delta_t < 0$, under-estimates $\delta_t > 0$ — so over a batch they *cancel*: a critic wrong in both directions would report a loss of zero.

Training the Critic: Why the *Squared* Error?

The critic is trained by gradient descent on the *squared* TD error — the DQN loss $(y_t - Q)^2$, applied to V :

$$\mathcal{L}(\phi) = \mathbb{E}_t[\delta_t^2] = \mathbb{E}_t \left[\underbrace{(r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))}_{\text{target } y_t}^2 \right]$$

- ▶ **Why not just δ_t ?** It is signed — over-estimates give $\delta_t < 0$, under-estimates $\delta_t > 0$ — so over a batch they *cancel*: a critic wrong in both directions would report a loss of zero.
- ▶ **Why not $|\delta_t|$?** Non-negative too, but its gradient has constant magnitude (a tiny error and a huge one pull equally hard) and is undefined at 0. Squaring gives $\nabla_\phi \mathcal{L} = -2 \mathbb{E}[\delta_t \nabla_\phi V_\phi(s_t)]$: **the correction scales with the size of the mistake** — the “nudge by $\alpha \delta_t$ ” rule above.

Training the Critic: Why the *Squared* Error?

The critic is trained by gradient descent on the *squared* TD error — the DQN loss $(y_i - Q)^2$, applied to V :

$$\mathcal{L}(\phi) = \mathbb{E}_t[\delta_t^2] = \mathbb{E}_t \left[\underbrace{(r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t))}_{\text{target } y_t}^2 \right]$$

- ▶ **Why not just δ_t ?** It is signed — over-estimates give $\delta_t < 0$, under-estimates $\delta_t > 0$ — so over a batch they *cancel*: a critic wrong in both directions would report a loss of zero.
- ▶ **Why not $|\delta_t|$?** Non-negative too, but its gradient has constant magnitude (a tiny error and a huge one pull equally hard) and is undefined at 0. Squaring gives $\nabla_\phi \mathcal{L} = -2 \mathbb{E}[\delta_t \nabla_\phi V_\phi(s_t)]$: **the correction scales with the size of the mistake** — the “nudge by $\alpha \delta_t$ ” rule above.
- ▶ **The deeper reason:** $V^\pi(s)$ is an expectation, and the minimizer of a squared error is exactly the mean of the target — squared error is the loss whose optimum is the quantity we want. ($|\delta_t|$ would converge to the *median* return.)

- ▶ We introduced δ_t as the critic's *training* signal. Here is the second, less obvious role it plays — the one that makes actor-critic work:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (\text{the TD residual})$$

- ▶ When $V_\phi = V^\pi$ (a perfect critic), the **expected** TD residual equals the advantage:

$$\begin{aligned} \mathbb{E}[\delta_t \mid s_t, a_t] &= r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1}}[V^\pi(s_{t+1})] - V^\pi(s_t) \\ &= Q^\pi(s_t, a_t) - V^\pi(s_t) = A^\pi(s_t, a_t) \end{aligned}$$

where $r(s_t, a_t) = \mathbb{E}[r_t \mid s_t, a_t]$ is the expected reward function. In practice we use the observed r_t and a learned V_ϕ , so δ_t is an approximation of A^π .

- ▶ We introduced δ_t as the critic's *training* signal. Here is the second, less obvious role it plays — the one that makes actor-critic work:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (\text{the TD residual})$$

- ▶ When $V_\phi = V^\pi$ (a perfect critic), the **expected** TD residual equals the advantage:

$$\begin{aligned}\mathbb{E}[\delta_t \mid s_t, a_t] &= r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1}}[V^\pi(s_{t+1})] - V^\pi(s_t) \\ &= Q^\pi(s_t, a_t) - V^\pi(s_t) = A^\pi(s_t, a_t)\end{aligned}$$

where $r(s_t, a_t) = \mathbb{E}[r_t \mid s_t, a_t]$ is the expected reward function. In practice we use the observed r_t and a learned V_ϕ , so δ_t is an approximation of A^π .

- ▶ So we can use δ_t as a **low-variance, online** estimate of A^π — no full trajectory needed
- ▶ This is what REINFORCE left open: we now have a way to estimate A^π at every timestep
- ▶ **One number, two jobs:** δ_t trains the critic (via $\mathcal{L}(\phi) = \mathbb{E}_t[\delta_t^2]$) *and* weights the actor's gradient. That is the whole actor-critic loop.

```
Initialise actor  $\theta$ , critic  $\phi$ ;  
for each episode do  
  observe initial state  $s_0$ ;  
  for  $t = 0, 1, 2, \dots$  do  
    sample  $a_t \sim \pi_\theta(\cdot | s_t)$ ;  
    take  $a_t$ , observe  $r_t, s_{t+1}$ ;  
     $\delta_t \leftarrow r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ ;  
     $\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \log \pi_\theta(a_t | s_t) \delta_t$ ;  
     $\phi \leftarrow \phi - \alpha_\phi \delta_t \nabla_\phi V_\phi(s_t)$ ;  
  end  
end
```

Algorithm: One-step Actor-Critic (online)

- ▶ **Actor** steps along the policy gradient, weighted by δ_t
- ▶ **Critic** regresses $V_\phi(s_t)$ toward $r_t + \gamma V_\phi(s_{t+1})$
- ▶ The same δ_t drives *both* updates — no full trajectory needed
- ▶ Note the signs: the actor *ascends* (+) to maximize return; the critic *descends* (−) to minimize $\mathbb{E}[\delta_t^2]$
- ▶ This is the pure **1-step TD** form. It is complete and it works — but the single δ_t is a crude advantage estimate, which is what we fix next

- ▶ **Variance reduction vs REINFORCE:** the TD residual δ_t is a much lower-variance signal than REINFORCE's reward-to-go minus baseline, $R_t - b(s_t)$, because it bootstraps off V_ϕ rather than waiting for the full trajectory
- ▶ **Online updates:** no need to wait until the end of an episode — the critic provides a training signal at every timestep
- ▶ **Bootstrapping:** propagating value estimates forward allows the agent to learn from partial trajectories and even in continuing (non-episodic) tasks

- ▶ **Variance reduction vs REINFORCE:** the TD residual δ_t is a much lower-variance signal than REINFORCE's reward-to-go minus baseline, $R_t - b(s_t)$, because it bootstraps off V_ϕ rather than waiting for the full trajectory
 - ▶ **Online updates:** no need to wait until the end of an episode — the critic provides a training signal at every timestep
 - ▶ **Bootstrapping:** propagating value estimates forward allows the agent to learn from partial trajectories and even in continuing (non-episodic) tasks
 - ▶ **Trade-off:** if V_ϕ is a bad critic, the TD residual is a biased advantage estimate — the actor is being guided by a noisy critic
 - ▶ Both networks must be trained together carefully; instability can arise if the critic lags the actor
- ⇒ So we have swapped REINFORCE's *variance* problem for a *bias* problem. Neither extreme is right — the next section builds the dial between them.

Policy Optimization: **Better Advantage Estimates**

The Question We Have Been Dodging: How Far Do We Look Ahead?

The actor's update is only as good as the number $\hat{A}_t$ we multiply it by. We have met **two** ways to produce that number, and they are opposite extremes:

Monte Carlo (REINFORCE)

$$\hat{A}_t = \underbrace{R_t}_{\text{all real rewards}} - V_\phi(s_t)$$
$$R_t = \sum_{t' \geq t} \gamma^{t'-t} r_{t'} \text{ (reward-to-go)}$$

Never trusts the critic beyond s_t .

⇒ **unbiased**, but every random event in the rest of the episode is baked into the number:
huge variance.

1-step TD (this lecture)

$$\hat{A}_t = \delta_t = \underbrace{r_t}_{\text{1 real reward}} + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$$

Trusts the critic completely after one step.

⇒ **low variance**, but inherits every mistake V_ϕ makes: **biased**.

- Neither is obviously right — and *which* is better changes **during training**: a freshly initialised critic is worthless (trust the real rewards), a well-trained one is excellent (trust the critic, keep the variance low).

- **Concrete failure of 1-step TD:** a robot knocks over a cup at step t , but the penalty only arrives at step $t+30$. The 1-step residual δ_t can only see that penalty *through* $V_\phi(s_{t+1})$. If the critic has not learned about the delayed penalty yet, $\delta_t \approx 0$ — the actor is told the disastrous action was fine.

- ▶ **Concrete failure of 1-step TD:** a robot knocks over a cup at step t , but the penalty only arrives at step $t+30$. The 1-step residual δ_t can only see that penalty *through* $V_\phi(s_{t+1})$. If the critic has not learned about the delayed penalty yet, $\delta_t \approx 0$ — the actor is told the disastrous action was fine.
- ▶ **Concrete failure of Monte Carlo:** the same robot later gets a random $+100$ from an unrelated event at step $t+200$. That windfall is added to $\hat{A}_t$ for *every* earlier action, good or bad — pure noise drowning the real signal.

- ▶ **Concrete failure of 1-step TD:** a robot knocks over a cup at step t , but the penalty only arrives at step $t+30$. The 1-step residual δ_t can only see that penalty *through* $V_\phi(s_{t+1})$. If the critic has not learned about the delayed penalty yet, $\delta_t \approx 0$ — the actor is told the disastrous action was fine.
 - ▶ **Concrete failure of Monte Carlo:** the same robot later gets a random $+100$ from an unrelated event at step $t+200$. That windfall is added to $\hat{A}_t$ for *every* earlier action, good or bad — pure noise drowning the real signal.
- ⇒ The real knob is: **how many real rewards do we look at before handing off to the critic?**
- ▶ One step is one answer. The whole episode is another. **Everything in between is also allowed** — and that family is what the next slide builds.
 - ▶ **Generalized Advantage Estimation (GAE)** is the standard way to navigate this family. It is not an exotic add-on: PPO, A2C, and nearly every modern on-policy implementation use it.

- Between 1-step TD and full Monte Carlo lies a whole family of **n -step advantage estimators**:

$$\hat{A}_t^{(n)} = \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V_\phi(s_{t+n})}_{n\text{-step return}} - V_\phi(s_t)$$

- Take n real rewards, then hand off to the critic. Small n : low variance, high bias; large n : high variance, low bias
- $n = 1$ recovers δ_t ; $n = \infty$ recovers Monte Carlo — our two extremes are just members of this family

- ▶ Between 1-step TD and full Monte Carlo lies a whole family of **n -step advantage estimators**:

$$\hat{A}_t^{(n)} = \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V_\phi(s_{t+n})}_{n\text{-step return}} - V_\phi(s_t)$$

- Take n real rewards, then hand off to the critic. Small n : low variance, high bias; large n : high variance, low bias
 - $n = 1$ recovers δ_t ; $n = \infty$ recovers Monte Carlo — our two extremes are just members of this family
- ▶ Each $\hat{A}_t^{(n)}$ telescopes into a sum of TD residuals we already compute:

$$\hat{A}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l \delta_{t+l}$$

- ▶ **So which n ?** Any single choice is a hard cutoff — full trust in real rewards up to step n , then abrupt total trust in the critic — and it is one more hyperparameter to tune per task.

- **Generalized Advantage Estimation** takes a **weighted average of every** $\hat{A}_t^{(n)}$, with weights decaying geometrically at rate $\lambda \in [0, 1]$ so shorter (lower-variance) estimates count most:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

- **Generalized Advantage Estimation** takes a **weighted average of every** $\hat{A}_t^{(n)}$, with weights decaying geometrically at rate $\lambda \in [0, 1]$ so shorter (lower-variance) estimates count most:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \hat{A}_t^{(n)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

- **Why an average helps:** no cliff at step n — the handover from real rewards to the critic is gradual, and no single n has to be right.
- **Why it is free:** the average looks expensive but — via the telescoping on the previous slide — collapses into **one discounted sum of the TD residuals we already have**. Same cost as 1-step TD.
- λ smoothly tunes the bias–variance trade-off (next slide: the limits)

- ▶ The single knob λ in $\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}$ interpolates the bias–variance trade-off:
- ▶ $\lambda = 0$: pure 1-step TD — low variance, biased
- ▶ $\lambda = 1$: full Monte Carlo $(R_t - V_\phi(s_t))$ — unbiased, high variance
- ▶ $\lambda \approx 0.95$: standard in PPO implementations
- ▶ **Read λ as “how much do I trust my critic?”** — small λ leans on V_ϕ , large λ leans on the observed rewards

- ▶ The single knob λ in $\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}$ interpolates the bias–variance trade-off:
- ▶ $\lambda = 0$: pure 1-step TD — low variance, biased
- ▶ $\lambda = 1$: full Monte Carlo $(R_t - V_\phi(s_t))$ — unbiased, high variance
- ▶ $\lambda \approx 0.95$: standard in PPO implementations
- ▶ **Read λ as “how much do I trust my critic?”** — small λ leans on V_ϕ , large λ leans on the observed rewards
- ▶ **Computed backwards in one pass** over a rollout:

$$\hat{A}_t = \delta_t + \gamma \lambda \hat{A}_{t+1}, \quad \hat{A}_T = 0$$

- ▶ We write $\hat{A}_t$ for short: our **practical estimate** of the true advantage $A^\pi(s_t, a_t)$

- ▶ **Key idea:** run N parallel actors sharing the same θ and ϕ
- ▶ All actors step simultaneously; gradients are aggregated and a single synchronous update is applied
- ▶ The shared critic sees diverse experience from all environments — more stable training signal
- ▶ **Policy loss:** $\mathcal{L}^{\pi}(\theta) = -\mathbb{E}_t \left[\log \pi_{\theta}(a_t | s_t) \hat{A}_t \right]$
- ▶ **Value loss:** $\mathcal{L}^V(\phi) = \mathbb{E}_t [\delta_t^2]$
- ▶ A2C is what most modern Actor-Critic implementations actually use
- ▶ Parallel actors de-correlate experience without a replay buffer (on-policy)

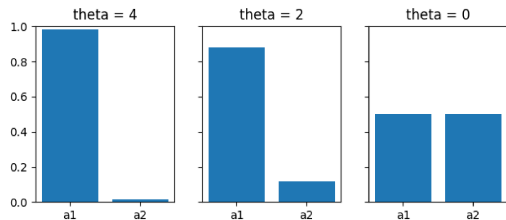
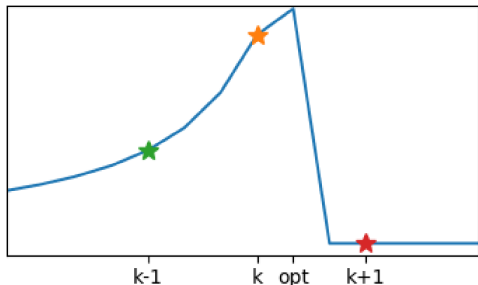
- ▶ **A3C** (Mnih et al., 2016): Asynchronous Advantage Actor-Critic
- ▶ Workers run asynchronously on CPU threads, applying gradients to a shared model without locks (Hogwild-style updates)
- ▶ The asynchrony acts as an implicit decorrelation mechanism — each worker is at a different point in its trajectory

- ▶ **A3C** (Mnih et al., 2016): Asynchronous Advantage Actor-Critic
- ▶ Workers run asynchronously on CPU threads, applying gradients to a shared model without locks (Hogwild-style updates)
- ▶ The asynchrony acts as an implicit decorrelation mechanism — each worker is at a different point in its trajectory
- ▶ **In practice:** A2C (synchronous) performs at least as well as A3C and is simpler to implement and reproduce
- ▶ A3C's asynchrony introduces **gradient staleness**: a worker may apply a gradient computed under an older version of θ

- ▶ **A3C** (Mnih et al., 2016): Asynchronous Advantage Actor-Critic
- ▶ Workers run asynchronously on CPU threads, applying gradients to a shared model without locks (Hogwild-style updates)
- ▶ The asynchrony acts as an implicit decorrelation mechanism — each worker is at a different point in its trajectory
- ▶ **In practice:** A2C (synchronous) performs at least as well as A3C and is simpler to implement and reproduce
- ▶ A3C's asynchrony introduces **gradient staleness**: a worker may apply a gradient computed under an older version of θ
- ▶ **Going forward:** we use A2C as the basis for PPO; A3C is historical context

- ▶ **A3C** (Mnih et al., 2016): Asynchronous Advantage Actor-Critic
- ▶ Workers run asynchronously on CPU threads, applying gradients to a shared model without locks (Hogwild-style updates)
- ▶ The asynchrony acts as an implicit decorrelation mechanism — each worker is at a different point in its trajectory
- ▶ **In practice:** A2C (synchronous) performs at least as well as A3C and is simpler to implement and reproduce
- ▶ A3C's asynchrony introduces **gradient staleness**: a worker may apply a gradient computed under an older version of θ
- ▶ **Going forward:** we use A2C as the basis for PPO; A3C is historical context
- ▶ But parallel actors only *accelerate* data collection — they do not fix the underlying issue that **a single gradient step can still destroy the policy**. That motivates the next section.

Why Step Size Matters



- ▶ A large gradient step in *parameter space* can cause a catastrophically large shift in *policy space*
- ▶ Once a policy collapses (e.g., always picks one bad action), the agent collects bad data and the policy is hard to recover
- ▶ We need to control how much the policy distribution changes, not just how much θ changes

- ▶ Last slide's conclusion: *don't let the policy move too far in one update*. But to act on that we first need to answer a prior question:

Given a candidate π_θ , how much better or worse is it than $\pi_{\theta_{\text{old}}}$ — without running it?

- ▶ Why we can't just run it: evaluating $\mathcal{J}(\theta)$ means collecting a fresh batch of rollouts for every candidate θ we consider. That is prohibitively expensive — and if the candidate is bad, we only find out by damaging the agent.
- ⇒ So we need a **score we can compute from the batch we already collected** — a stand-in for $\mathcal{J}(\theta)$, called a **surrogate objective**. Next slide builds it from scratch.

What the batch contains: for every timestep, the state s_t , the action a_t we took, and $\hat{A}_t$ — whether that action turned out better (+) or worse (−) than usual.

- ▶ A candidate π_θ cannot change what happened. The *only* thing it changes is **how likely each of those actions would have been**.

¹ $r_t(\theta)$ is the probability ratio, not the reward r_t from earlier lectures — distinguished by the θ argument. 

What the batch contains: for every timestep, the state s_t , the action a_t we took, and $\hat{A}_t$ — whether that action turned out better (+) or worse (−) than usual.

- ▶ A candidate π_θ cannot change what happened. The *only* thing it changes is **how likely each of those actions would have been**.
- ▶ So the question collapses to: *does π_θ put more probability on the actions that went well, and less on the ones that went badly?*

¹ $r_t(\theta)$ is the probability ratio, not the reward r_t from earlier lectures — distinguished by the θ argument.

What the batch contains: for every timestep, the state s_t , the action a_t we took, and $\hat{A}_t$ — whether that action turned out better (+) or worse (−) than usual.

- ▶ A candidate π_θ cannot change what happened. The *only* thing it changes is **how likely each of those actions would have been**.
- ▶ So the question collapses to: *does π_θ put more probability on the actions that went well, and less on the ones that went badly?*
- ▶ Measure that per action with the **probability ratio**:¹

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

1 : would take a_t just as often
2 : twice as often 0.5 : half as often

¹ $r_t(\theta)$ is the probability ratio, not the reward r_t from earlier lectures — distinguished by the θ argument. 

What the batch contains: for every timestep, the state s_t , the action a_t we took, and $\hat{A}_t$ — whether that action turned out better (+) or worse (−) than usual.

- ▶ A candidate π_θ cannot change what happened. The *only* thing it changes is **how likely each of those actions would have been**.
- ▶ So the question collapses to: *does π_θ put more probability on the actions that went well, and less on the ones that went badly?*
- ▶ Measure that per action with the **probability ratio**:¹

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

1 : would take a_t just as often
2 : twice as often 0.5 : half as often

- ▶ Score each sample by **(how much more the new policy wants it) × (how good it was)**, and average:

$$\mathcal{L}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right]$$

¹ $r_t(\theta)$ is the probability ratio, not the reward r_t from earlier lectures — distinguished by the θ argument. 

Every column is a number we can just compute: $\hat{A}_t$ from the critic, the probabilities by evaluating each network on (s_t, a_t) .

	$\hat{A}_t$	$\pi_{\theta_{\text{old}}}(a_t s_t)$	$\pi_{\theta}(a_t s_t)$	$r_t(\theta)$	$r_t(\theta) \hat{A}_t$
went well	+2.0	0.20	0.40	2.0	+4.0
went badly	-1.0	0.50	0.40	0.8	-0.8

- $\mathcal{L}(\theta) = \frac{1}{2}(4.0 - 0.8) = +1.6 > 0$: the candidate **doubles down on what worked, backs off what didn't**. Ascending $\mathcal{L}$ searches for exactly that.

Every column is a number we can just compute: $\hat{A}_t$ from the critic, the probabilities by evaluating each network on (s_t, a_t) .

	$\hat{A}_t$	$\pi_{\theta_{\text{old}}}(a_t s_t)$	$\pi_{\theta}(a_t s_t)$	$r_t(\theta)$	$r_t(\theta) \hat{A}_t$
went well	+2.0	0.20	0.40	2.0	+4.0
went badly	-1.0	0.50	0.40	0.8	-0.8

- ▶ $\mathcal{L}(\theta) = \frac{1}{2}(4.0 - 0.8) = +1.6 > 0$: the candidate **doubles down on what worked, backs off what didn't**. Ascending $\mathcal{L}$ searches for exactly that.
- ▶ **Why the ratio is the right multiplier**: the batch was collected by the *old* policy, so each action appears about as often as *it* would have chosen it. If the new policy would pick that action twice as often, the sample should count twice. That re-counting is all the ratio does.

Every column is a number we can just compute: $\hat{A}_t$ from the critic, the probabilities by evaluating each network on (s_t, a_t) .

	$\hat{A}_t$	$\pi_{\theta_{\text{old}}}(a_t s_t)$	$\pi_{\theta}(a_t s_t)$	$r_t(\theta)$	$r_t(\theta) \hat{A}_t$
went well	+2.0	0.20	0.40	2.0	+4.0
went badly	-1.0	0.50	0.40	0.8	-0.8

- ▶ $\mathcal{L}(\theta) = \frac{1}{2}(4.0 - 0.8) = +1.6 > 0$: the candidate **doubles down on what worked, backs off what didn't**. Ascending $\mathcal{L}$ searches for exactly that.
- ▶ **Why the ratio is the right multiplier**: the batch was collected by the *old* policy, so each action appears about as often as *it* would have chosen it. If the new policy would pick that action twice as often, the sample should count twice. That re-counting is all the ratio does.
- ▶ At $\theta = \theta_{\text{old}}$ every $r_t = 1$, so $\mathcal{L}$ starts at ≈ 0 and its gradient is exactly the vanilla policy gradient. **This is the objective PPO maximizes.**

$\mathcal{L}(\theta)$ is an *approximation* of the true improvement, and it degrades in two ways as π_θ moves away from $\pi_{\theta_{\text{old}}}$:

1. The re-weighting becomes statistically useless

If $\pi_{\theta_{\text{old}}}(a|s)$ is tiny where $\pi_\theta(a|s)$ is large, $r_t(\theta)$ blows up: one rare sample gets an enormous weight and swings the entire gradient. The estimate stays unbiased but its **variance explodes**.

2. We quietly swapped the state distribution

Importance sampling corrects for the *actions*, but we still evaluate them at the states $\pi_{\theta_{\text{old}}}$ happened to visit. A very different policy visits very different states — so the surrogate scores the new policy on a world it will no longer see.

► Both errors are **small when the two policies are close**, and unbounded when they are not.

⇒ This is the principled version of “big steps break things”. Everything that follows — TRPO and PPO — is one idea: **maximize $\mathcal{L}(\theta)$ while keeping π_θ close to $\pi_{\theta_{\text{old}}}$** , differing only in how “close” is enforced.

Policy Optimization: **TRPO**

TRPO writes “improve, but don't move far” as a **constrained optimization problem**:

$$\underbrace{\max_{\theta} \mathcal{L}(\theta)}_{\text{what we want}} \quad \underbrace{\text{s.t.}}_{\text{“subject to”}} \quad \underbrace{\mathbb{E}_t[D_{KL}(\pi_{\theta_{\text{old}}}(\cdot|s_t) \parallel \pi_{\theta}(\cdot|s_t))]}_{\text{the rule the answer must obey}} \leq \delta$$

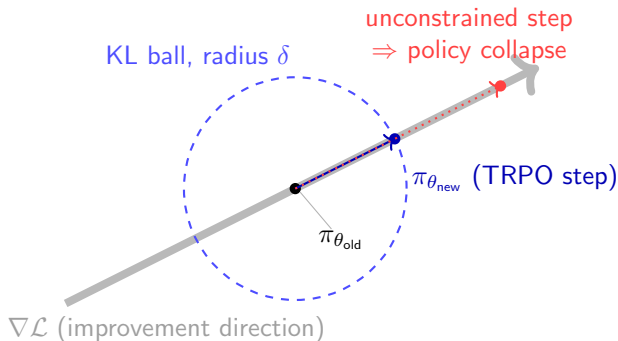
- ▶ **Reading it:** “pick the θ that maximizes the surrogate $\mathcal{L}$, *among only those* θ whose policy stays within δ of the old one.” Without the constraint the maximizer is unbounded — $\mathcal{L}$ keeps rewarding a bigger ratio.
- ▶ $D_{KL}(p \parallel q)$ (**KL divergence**) measures how different two probability distributions are: 0 when identical, growing as they disagree. Here it compares the old and new *action distributions* in the same state s_t — so it measures change in **behaviour**, not in weights.
- ▶ δ is our **step-size budget**²— a small number we choose (typically 0.01), the radius of the **trust region**. Bigger δ = faster but riskier updates.

²Unrelated to the TD residual δ_t from earlier today — unfortunate but standard notation; the KL bound never carries a time subscript.

- ▶ The constraint is on **behaviour** (“the policy must not change much”), but the only thing we can actually turn is the **weights**. Those are not the same thing: some weights barely affect what the agent does, others flip it completely.
- ▶ So before every update TRPO has to work out the **conversion rate** between “changed the weights this much” and “changed the behaviour this much” — that conversion table is the **Fisher information matrix**.

- ▶ The constraint is on **behaviour** (“the policy must not change much”), but the only thing we can actually turn is the **weights**. Those are not the same thing: some weights barely affect what the agent does, others flip it completely.
- ▶ So before every update TRPO has to work out the **conversion rate** between “changed the weights this much” and “changed the behaviour this much” — that conversion table is the **Fisher information matrix**.
- ▶ **The problem:** that table has one entry for every *pair* of weights. A modest 1M-parameter network needs 10^{12} of them — it cannot be built, let alone solved.
- ▶ TRPO has clever machinery to avoid ever building it, but each update still costs several extra passes over the data plus a search to check the constraint was respected.

- ▶ The constraint is on **behaviour** (“the policy must not change much”), but the only thing we can actually turn is the **weights**. Those are not the same thing: some weights barely affect what the agent does, others flip it completely.
 - ▶ So before every update TRPO has to work out the **conversion rate** between “changed the weights this much” and “changed the behaviour this much” — that conversion table is the **Fisher information matrix**.
 - ▶ **The problem:** that table has one entry for every *pair* of weights. A modest 1M-parameter network needs 10^{12} of them — it cannot be built, let alone solved.
 - ▶ TRPO has clever machinery to avoid ever building it, but each update still costs several extra passes over the data plus a search to check the constraint was respected.
- ⇒ **Great theory, painful engineering:** hundreds of lines of second-order code for one policy update. *PPO's question: can we get the same “don't move too far” effect with a one-line change to the loss?*



- Each update stays within a **KL ball** around $\pi_{\theta_{old}}$ (*policy space*, not parameter space): the step climbs $\mathcal{L}$ but is clipped at the boundary, preventing the overshoot that collapses the policy

Policy Optimization: **PPO**

- ▶ TRPO achieves monotonic improvement but requires second-order optimisation (conjugate gradient, Fisher matrix) — expensive and complex to implement
- ▶ **Question:** can we get TRPO-level stability with plain SGD?

- ▶ TRPO achieves monotonic improvement but requires second-order optimisation (conjugate gradient, Fisher matrix) — expensive and complex to implement
- ▶ **Question:** can we get TRPO-level stability with plain SGD?
- ▶ **PPO** (Schulman et al., 2017) proposes two variants:
 - **PPO-KL:** add a KL penalty to the objective and adapt the penalty coefficient β over training
 - **PPO-Clip:** clip the probability ratio $r_t(\theta)$ to prevent large policy updates
- ▶ PPO-Clip is the standard variant — simpler, no hyperparameter tuning on β , works at least as well empirically

► Recall the probability ratio: $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$

► The **PPO-Clip objective** is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

► ε is a small hyperparameter (typically $\varepsilon = 0.2$)

► Recall the probability ratio: $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$

► The **PPO-Clip objective** is:

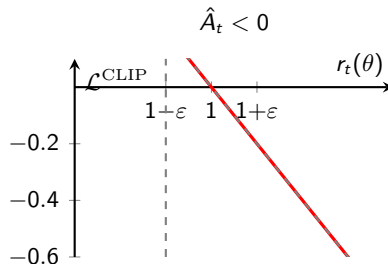
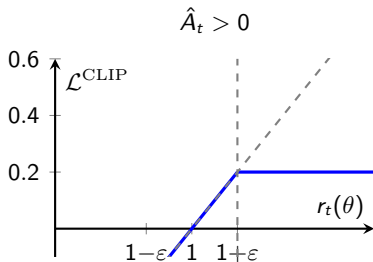
$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

► ε is a small hyperparameter (typically $\varepsilon = 0.2$)

► **Why clip + min?**

- When $\hat{A}_t > 0$: clipping caps r_t at $1 + \varepsilon$, removing the incentive to increase the ratio further
- When $\hat{A}_t < 0$: clipping floors r_t at $1 - \varepsilon$, removing the incentive to decrease the ratio further
- The **min** ensures the objective is a lower bound — we never benefit from going outside $[1 - \varepsilon, 1 + \varepsilon]$

► This enforces a soft trust region without any second-order computation



- ▶ **Left ($\hat{A}_t > 0$):** benefit grows linearly with r_t until $1 + \epsilon$, then is clipped flat — no reward for pushing the policy further
- ▶ **Right ($\hat{A}_t < 0$):** penalty grows as r_t decreases until $1 - \epsilon$, then is clipped flat
- ▶ Gray dashed line = unclipped surrogate; colored solid = clipped (what PPO optimises)

Algorithm 5 PPO with Clipped Objective

Input: initial policy parameters θ_0 , clipping threshold ϵ

for $k = 0, 1, 2, \dots$ **do**

Collect set of partial trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Compute policy update

$$\theta_{k+1} = \arg \max_{\theta} \mathcal{L}_{\theta_k}^{CLIP}(\theta)$$

by taking K steps of minibatch SGD (via Adam), where

$$\mathcal{L}_{\theta_k}^{CLIP}(\theta) = \mathbb{E}_{\tau \sim \pi_k} \left[\sum_{t=0}^T \left[\min(r_t(\theta) \hat{A}_t^{\pi_k}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^{\pi_k}) \right] \right]$$

end for

- ▶ **Sample efficiency:** PPO runs multiple SGD epochs over the same rollout — unlike A2C, which discards data after one update
- ▶ GAE is used to compute $\hat{A}_t$ once per rollout, before the epoch loop

Actor and critic share one optimizer step, so their objectives are added into a single number to differentiate — that is all “combined loss” means:

$$\mathcal{L}(\theta, \phi) = \underbrace{\mathcal{L}^{\text{CLIP}}(\theta)}_{\text{1. make good actions likelier}} - c_1 \underbrace{\mathcal{L}^V(\phi)}_{\text{2. train the critic}} + c_2 \underbrace{\mathcal{H}(\pi_\theta)}_{\text{3. stay exploratory}}$$

- ▶ 1. $\mathcal{L}^{\text{CLIP}}$ — the clipped surrogate we just built. *Maximized*, so it enters with +.
- ▶ 2. $\mathcal{L}^V = \mathbb{E}_t[\delta_t^2]$ — the critic’s squared TD error. We *minimize* it, hence the minus sign: subtracting it from a quantity we maximize is the same as minimizing it. $c_1 \approx 0.5$ balances the two scales.

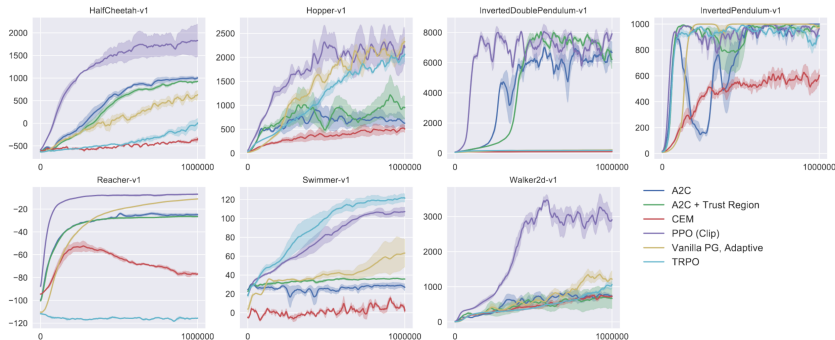
Actor and critic share one optimizer step, so their objectives are added into a single number to differentiate — that is all “combined loss” means:

$$\mathcal{L}(\theta, \phi) = \underbrace{\mathcal{L}^{\text{CLIP}}(\theta)}_{\text{1. make good actions likelier}} - c_1 \underbrace{\mathcal{L}^V(\phi)}_{\text{2. train the critic}} + c_2 \underbrace{\mathcal{H}(\pi_\theta)}_{\text{3. stay exploratory}}$$

- ▶ **1.** $\mathcal{L}^{\text{CLIP}}$ — the clipped surrogate we just built. *Maximized*, so it enters with $+$.
- ▶ **2.** $\mathcal{L}^V = \mathbb{E}_t[\delta_t^2]$ — the critic’s squared TD error. We *minimize* it, hence the minus sign: subtracting it from a quantity we maximize is the same as minimizing it. $c_1 \approx 0.5$ balances the two scales.
- ▶ **3.** $\mathcal{H}(\pi_\theta)$ — the **entropy** of the action distribution: high when the policy is still undecided, near zero when it always picks one action. Adding it *rewards* indecision, which stops the policy from collapsing onto one action before it has explored. c_2 is small (≈ 0.01); the term reappears as a first-class citizen in SAC.
- ▶ In code this is three lines summed and passed to one `loss.backward()` — no separate actor and critic updates.

- ▶ The clipped objective is only half the story — these implementation details matter *as much as* ε , and explain why two “correct” PPO implementations can differ by an order of magnitude³
- ▶ **Advantage normalisation:** standardise $\hat{A}_t$ per minibatch (stabilises gradient scale)
- ▶ **Value-loss clipping:** clip the critic update like the policy
- ▶ **Gradient clipping:** bound global grad-norm (typically 0.5)
- ▶ **KL early stopping:** end the epoch loop once mean KL exceeds a target — a cheap trust-region safeguard
- ▶ **Obs / reward normalisation:** running normalisation keeps inputs and returns well-scaled
- ▶ **Orthogonal init:** small policy-head gain avoids an over-confident initial policy

³Andrychowicz et al. (2020), *What Matters in On-Policy RL?*



- PPO matches or exceeds TRPO with far less complexity; outperforms vanilla PG and CEM on most benchmarks⁴

⁴Schulman, Wolski, Dhariwal, Radford, Klimov, 2017

Policy Optimization: **GRPO**

- ▶ The fragile part of the Actor-Critic loop is the **critic** V_ϕ — training two networks together is the main source of instability
- ▶ **Group Relative Policy Optimization** (GRPO; Shao et al., 2024) removes the critic entirely

- ▶ The fragile part of the Actor-Critic loop is the **critic** V_ϕ — training two networks together is the main source of instability
- ▶ **Group Relative Policy Optimization** (GRPO; Shao et al., 2024) removes the critic entirely
- ▶ Per prompt/state, sample a **group** of G outputs under $\pi_{\theta_{\text{old}}}$, scored with rewards R_i
- ▶ Use the **group mean as the baseline** — a Monte Carlo advantage, no value function:

$$\hat{A}_i = \frac{R_i - \overbrace{\text{mean}(R_1, \dots, R_G)}^{\text{the "Group Relative" baseline}}}{\underbrace{\text{std}(R_1, \dots, R_G)}_{\text{per-group advantage normalisation}}}$$

- ▶ Only the **mean-subtraction** is the GRPO contribution — that is the “Group Relative” baseline replacing the critic V_ϕ
- ▶ The **std-normalisation** is a *separate* stabilisation trick: it is exactly the per-minibatch advantage normalisation from the PPO-in-Practice slide, applied per group — not part of the baseline
- ▶ So a Monte Carlo baseline does *not* “need” variance correction; the std term is an orthogonal, optional stabiliser

- ▶ Only the **mean-subtraction** is the GRPO contribution — that is the “Group Relative” baseline replacing the critic V_ϕ
- ▶ The **std-normalisation** is a *separate* stabilisation trick: it is exactly the per-minibatch advantage normalisation from the PPO-in-Practice slide, applied per group — not part of the baseline
- ▶ So a Monte Carlo baseline does *not* “need” variance correction; the std term is an orthogonal, optional stabiliser
- ▶ Plug $\hat{A}_i$ into the *same* PPO-Clip objective (plus a KL term to a reference policy)
- ▶ **Why it matters:** no critic $\Rightarrow$ no two-network instability and $\approx$ half the memory
- ▶ Since 2024 GRPO is the default for **LLM RL** (DeepSeek-R1, many open recipes) — revisited in **RLHF**, where the reward is a learned preference model

Policy Optimization: **Summary**

REINFORCE $\rightarrow$ +baseline $\rightarrow$ +critic (A2C) $\rightarrow$ +GAE $\rightarrow$ +trust region (TRPO) $\rightarrow$ +clipping (PPO) $\rightarrow$ -critic (GRPO)

Algorithm	On-policy?	Variance	Sample eff.	Complexity
REINFORCE	Yes	High	Low	Simple
A2C	Yes	Medium	Medium	Moderate
TRPO	Yes	Medium	Medium	High
PPO	Yes	Medium	Medium	Low
GRPO	Yes	Medium	Medium	Low (no critic)

- ▶ **TD learning** — update a prediction using a later, better-informed one. The TD error $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ is the same bracket you saw in Q-learning and SARSA; the critic is trained by minimising its *square*
- ▶ A **critic** V_ϕ turns REINFORCE's baseline into something learnable, and δ_t doubles as an online, low-variance estimate of the advantage — one number, two jobs
- ▶ **GAE** replaces the crude 1-step δ_t with a λ -weighted average over all lookahead lengths, computed backwards in one pass: $\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}$. Read λ as “how much do I trust my critic”
- ▶ $\hat{A}_t$ is what *every* algorithm from here on multiplies the log-probability gradient by
- ▶ **A2C** parallelises data collection with N synchronous actors; A3C uses asynchronous workers (empirically $A2C \geq A3C$)

- ▶ A large step in *weights* can be a catastrophic step in *behaviour* — and a collapsed policy collects bad data, so it rarely recovers
- ▶ To take a step safely we must score a candidate policy **without running it**. The **surrogate objective** $\mathcal{L}(\theta) = \mathbb{E}_t[r_t(\theta) \hat{A}_t]$ does this by re-counting already-collected samples by the ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$
- ▶ That score is only trustworthy while the two policies stay close — hence every method below is “maximize $\mathcal{L}$, but don’t move far”
- ▶ **TRPO** enforces this with a hard KL constraint. Correct, but it needs the Fisher matrix: second-order, expensive, painful to implement
- ▶ **PPO-Clip** gets the same effect first-order: clip $r_t(\theta)$ to $[1-\varepsilon, 1+\varepsilon]$ and take the min with the unclipped term. One combined loss = clipped objective $- c_1 \times$ critic error $+ c_2 \times$ entropy
- ▶ **GRPO** drops the critic entirely, using a group-mean baseline — now the default for LLM RL (DeepSeek-R1)

- ▶ **Function approximation bias:** the TD residual δ_t is only an unbiased advantage estimate if $V_\phi = V^\pi$. An inaccurate critic introduces bias into the policy gradient
- ▶ **Two-network instability:** if the critic lags behind the actor (or vice versa), the feedback loop can diverge — both networks must be trained carefully together (GRPO sidesteps this by dropping the critic entirely)
- ▶ **Hyperparameter sensitivity:** PPO performance is sensitive to ε (clip range), λ (GAE), number of epochs per rollout, and entropy coefficient c_2
- ▶ **On-policy sample inefficiency:** all methods in this lecture discard experience after each update. In complex environments this can require billions of environment steps
- ▶ **Dense-reward assumption:** variance reduction via advantage estimation works well when rewards are frequent. Sparse-reward tasks remain challenging for all on-policy methods

- ▶ **Key limitation:** on-policy methods discard all experience after each update — data-hungry for complex environments

- ▶ **Key limitation:** on-policy methods discard all experience after each update — data-hungry for complex environments
- ▶ **DDPG & TD3:** off-policy deterministic policies for continuous control — replay buffer, action-value critic $Q_\phi(s, a)$, gradient through $Q_\phi(s, \mu_\theta(s))$
- ▶ **SAC:** max-entropy Actor-Critic — the same $c_2 \mathcal{H}(\pi_\theta)$ bonus you saw in PPO, now **central** to the objective
- ▶ **Exploration:** that entropy bonus *is* exploration via a stochastic policy; later we make exploration the explicit objective
- ▶ **RLHF:** PPO is the algorithm underneath the original InstructGPT/ChatGPT pipeline; GRPO is its critic-free successor

Policy Optimization: **References**

- [1] Sutton, R. S., & Barto, A. G. (2018).

Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

(TD learning and the TD error: Ch. 6. n-step returns: Ch. 7. Actor-Critic: Ch. 13.)

- [2] Sutton, R. S., McAllester, D. A., Singh, S. P., & Mansour, Y. (2000).

Policy Gradient Methods for Reinforcement Learning with Function Approximation.

In *Advances in Neural Information Processing Systems (NeurIPS)* 12, 1057–1063.

(The policy gradient theorem behind REINFORCE's estimator, and the baseline/critic form.)

- [3] Kakade, S., & Langford, J. (2002).

Approximately Optimal Approximate Reinforcement Learning.

In *Proceedings of the 19th International Conference on Machine Learning (ICML)*, 267–274.

(*Performance Difference Lemma — the result behind the surrogate objective; see appendix.*)

- [4] Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016).

Asynchronous Methods for Deep Reinforcement Learning.

In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, PMLR 48, 1928–1937.

- [5] Schulman, J., Levine, S., Abbeel, P., Jordan, M., & Moritz, P. (2015).

Trust Region Policy Optimization.

In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, PMLR 37, 1889–1897.

(The KL trust region, and the Fisher-matrix machinery whose cost motivates PPO.)

- [6] Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016).

High-Dimensional Continuous Control Using Generalized Advantage Estimation.

In *4th International Conference on Learning Representations (ICLR)*. *arXiv:1506.02438*.

(GAE: the λ dial and the backward recursion $\hat{A}_t = \delta_t + \gamma\lambda\hat{A}_{t+1}$.)

- [7] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017).

Proximal Policy Optimization Algorithms.

arXiv preprint arXiv:1707.06347.

(PPO-Clip, and the combined clipped + value + entropy loss.)

- [8] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018).

Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.

In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, PMLR 80, 1861–1870.

- [9] Andrychowicz, M., Raichuk, A., Stańczyk, P., Orsini, M., Girgin, S., Marinier, R., et al. (2021).

What Matters In On-Policy Reinforcement Learning? A Large-Scale Empirical Study.
In *9th International Conference on Learning Representations (ICLR)*. *arXiv:2006.05990*.
(*The implementation details on the “PPO in Practice” slide.*)

- [10] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., et al. (2024).

DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.

arXiv preprint arXiv:2402.03300. (Introduces GRPO.)

[11] DeepSeek-AI. (2025).

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.
arXiv preprint arXiv:2501.12948.

[12] OpenAI Baselines.

<https://github.com/openai/baselines>

[13] Spinning Up by OpenAI.

<https://spinningup.openai.com>

[14] Sergey Levine, Berkeley CS285: Deep Reinforcement Learning.

<https://rail.eecs.berkeley.edu/deeprlcourse/>

[15] Chelsea Finn & Karol Hausman, Stanford CS224R: Deep Reinforcement Learning.

<http://cs224r.stanford.edu/>

[16] Emma Brunskill, Stanford CS234: Reinforcement Learning.

<https://web.stanford.edu/class/cs234/>

- ▶ The relative performance identity is the **Performance Difference Lemma**¹

$$\mathcal{J}(\theta) - \mathcal{J}(\theta_{\text{old}}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t \geq 0} \gamma^t A^{\pi_{\theta_{\text{old}}}}(s_t, a_t) \right]$$

- ▶ **Why it matters:** it makes the surrogate $\mathcal{L}(\theta)$ a **principled approximation** to the true objective $\mathcal{J}(\theta)$ — not a heuristic
- ▶ When $\pi_{\theta} \approx \pi_{\theta_{\text{old}}}$, the state distributions induced by the two policies are similar, so samples from $\pi_{\theta_{\text{old}}}$ can stand in for samples from π_{θ} with *bounded* error — TRPO makes this bound explicit in terms of the maximum KL divergence
- ▶ As π_{θ} drifts away, the bound degrades. This is the theoretical reason for a trust region

¹Lemma originally due to Kakade & Langford (2002); foundation of TRPO in Schulman et al. (2015).

- ▶ The IS-weighted estimator has variance

$$\text{Var}\left[\frac{P(x)}{Q(x)} f(x)\right] = \mathbb{E}_{x \sim P}\left[\frac{P(x)}{Q(x)} f(x)^2\right] - \mathbb{E}[f]^2$$

- ▶ It **explodes** when the ratio $\frac{P}{Q}$ is large where f^2 is large — a single bad sample dominates the estimate
- ▶ For **trajectory-level** IS: $\frac{p(\tau|\pi_\theta)}{p(\tau|\pi_{\theta_{\text{old}}})} = \prod_{t=0}^T r_t(\theta)$. Small per-step differences **compound multiplicatively** over T steps — the variance is hopeless
- ▶ This is why TRPO and PPO use **single-step ratios** and constrain them to stay near 1²

²Full derivation in OpenAI's **Spinning Up: TRPO** and Schulman et al. (2015).

Algorithm 3 Trust Region Policy Optimization

Input: initial policy parameters θ_0

for $k = 0, 1, 2, \dots$ **do**

Collect set of trajectories $\mathcal{D}_k$ on policy $\pi_k = \pi(\theta_k)$

Estimate advantages $\hat{A}_t^{\pi_k}$ using any advantage estimation algorithm

Form sample estimates for

- policy gradient $\hat{g}_k$ (using advantage estimates)
- and KL-divergence Hessian-vector product function $f(v) = \hat{H}_k v$

Use CG with n_{cg} iterations to obtain $x_k \approx \hat{H}_k^{-1} \hat{g}_k$

Estimate proposed step $\Delta_k \approx \sqrt{\frac{2\delta}{x_k^T \hat{H}_k x_k}} x_k$

Perform backtracking line search with exponential decay to obtain final update

$$\theta_{k+1} = \theta_k + \alpha^j \Delta_k$$

end for

- Reference only: the full TRPO algorithm (see Schulman et al., 2015)